

```
1 ### Cell 1: Imports
2 begin
3     using Pkg
4     Pkg.activate(".")
5     Pkg.instantiate()
6     using InfiniteOpt, Ipopt, Plots, NLSolve, LinearAlgebra
7     # optional: PlotlyJS; plotlyjs()
8 end
```

Activating project at `~/julia/pluto\_notebooks`

```
1 ### Cell 2: Parameters
2 begin
3     # Earth-Moon system
4      $\mu$  = 0.012150585609624 # Moon / (Earth + Moon)
5
6     const u_max = 0.1
7     const N_supports = 501
8     tf_lower = 1.0
9     tf_upper = 30.0
10    tf_start = 15.0
11
12    println(" $\mu$  = ",  $\mu$ , ", u_max = ", u_max, ", supports = ", N_supports)
13 end
```

μ = 0.012150585609624, u_max = 0.1, supports = 501

0.8369151257723574

```
1 ### Cell 3: Compute L1 (planar)
2 begin
3   function dOmega_dx(x, μ)
4     r1 = abs(x + μ)
5     r2 = abs(x - (1-μ))
6     return x - (1-μ)*(x + μ)/r1^3 - μ*(x - (1-μ))/r2^3
7   end
8   res = nlsolve(x -> dOmega_dx(x[1], μ), [0.84]; ftol = 1e-14)
9   if !NLSolve.converged(res)
10     @warn "L1 solver did not converge!"
11   end
12   x_L1 = res.zero[1]
13   println("Computed L1 x = ", x_L1)
14   x_L1
15 end
```

Computed L1 x = 0.8369151257723574

An InfiniteOpt Model
 Feasibility problem with:
 Finite parameters: 0
 Infinite parameter: 1
 Variables: 7
 Derivatives: 0
 Measures: 0
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
 Names registered in the model: tf, ux, uy, vx, vy, x, y, τ , $\partial\Omega x$, $\partial\Omega y$
 Transformation backend information:
 Backend type: TranscriptionBackend
 Solver: Ipopt
 Transformation built and up-to-date: false

```

1 ### Cell 4: Model + variables (purely planar)
2 begin
3   model = InfiniteModel(Ipopt.Optimizer)
4   @infinite_parameter(model,  $\tau \in [0, 1]$ , num_supports = N_supports)
5   @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)
6
7   # State variables (only in-plane)
8   @variable(model, x, Infinite( $\tau$ ), start =  $\tau \rightarrow (-\mu + 0.01)*(1 - \tau) + x_{L1} * \tau$ )
9   @variable(model, y, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
10  @variable(model, vx, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
11  @variable(model, vy, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
12
13  # Control: only x and y directions
14  @variable(model, -u_max <= ux <= u_max, Infinite( $\tau$ ))
15  @variable(model, -u_max <= uy <= u_max, Infinite( $\tau$ ))
16
17  # Helper functions (2D only)
18  r1(x_,y_) = sqrt((x_ +  $\mu$ )^2 + y_^2)
19  r2(x_,y_) = sqrt((x_ - (1- $\mu$ ))^2 + y_^2)
20   $\partial\Omega x(x_,y_) = x_ - (1-\mu)*(x_ + \mu)/r1(x_,y_)^3 - \mu*(x_ - (1-\mu))/r2(x_,y_)^3$ 
21   $\partial\Omega y(x_,y_) = y_ * (1-\mu)/r1(x_,y_)^3 + y_ * \mu / r2(x_,y_)^3$  # note sign!
22
23  model[: $\partial\Omega x$ ] =  $\partial\Omega x$ 
24  model[: $\partial\Omega y$ ] =  $\partial\Omega y$ 
25
26  model
27 end

```

$$tf \times \int_{\tau \in [0,1]} ux(\tau)^2 + uy(\tau)^2 d\tau$$

```

1 ### Cell 5: Dynamics + BCs + Objective
2 begin
3   # Planar CR3BP dynamics
4   @constraint(model, ∂(x, τ) == tf * vx)
5   @constraint(model, ∂(y, τ) == tf * vy)
6   @constraint(model, ∂(vx, τ) == tf * (2*vy + model[:∂Qx](x,y) + ux))
7   @constraint(model, ∂(vy, τ) == tf * (-2*vx + model[:∂Qy](x,y) + uy))
8
9   # Initial conditions: near Earth, zero velocity
10  @constraint(model, x(0) == -μ + 0.01)
11  @constraint(model, y(0) == 0.0)
12  @constraint(model, vx(0) == 0.0)
13  @constraint(model, vy(0) == 0.0)
14
15  # Final conditions: exactly at L1, zero velocity
16  @constraint(model, x(1) == x_L1)
17  @constraint(model, y(1) == 0.0)
18  @constraint(model, vx(1) == 0.0)
19  @constraint(model, vy(1) == 0.0)
20
21  # Minimum energy (L2-norm of thrust vector, scaled by tf for physical integral)
22  @objective(model, Min, tf * ∫(ux^2 + uy^2, τ))
23 end

```

```
1 ### Cell 6: Optimize
2 begin
3     @time optimize!(model)
4     println("Termination status: ", termination_status(model))
5     println("Solved tf = ", value(tf))
6 end
```

```
*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****
```

This is Ipopt version 3.14.19, running with linear solver MUMPS 5.8.1.

```
Number of nonzeros in equality constraint Jacobian...:   15026
Number of nonzeros in inequality constraint Jacobian.:      0
Number of nonzeros in Lagrangian Hessian.....:   519538

Total number of variables.....:   5011
    variables with only lower bounds:      0
    variables with lower and upper bounds:  1003
    variables with only upper bounds:      0
Total number of equality constraints.....:   4012
Total number of inequality constraints.....:      0
    inequality constraints with only lower bounds:      0
    inequality constraints with lower and upper bounds:  0
    inequality constraints with only upper bounds:      0
```

```
([-0.00215059, 0.146758, 0.253636, 0.338472, 0.409312, 0.470253, 0.522933, 0.568827, 0.608914
```

```
1 ### Cell 7: Extract solution
2 begin
3   opt_x = value(x)
4   opt_y = value(y)
5   opt_vx = value(vx)
6   opt_vy = value(vy)
7   opt_ux = value(ux)
8   opt_uy = value(uy)
9   opt_tf = value(tf)
10
11   ts = supports( $\tau$ ) .* opt_tf
12
13   println("Final position error: x = ", opt_x[end] - x_L1, " y = ", opt_y[end])
14   println("Final velocity error: vx = ", opt_vx[end], " vy = ", opt_vy[end])
15
16   solution = (opt_x, opt_y, opt_vx, opt_vy, opt_ux, opt_uy, ts, opt_tf)
17 end
```

```
Final position error: x = 0.0 y = -4.047771233132182e-19
Final velocity error: vx = 3.0054443220040726e-20 vy = 4.24560281133715e-19
```

```

1  ### Cell 8: Plots
2  begin
3
4      # Main trajectory plot
5      p_traj = plot(opt_x, opt_y, lw=2.5, label="Trajectory",
6                   xlabel="x", ylabel="y", aspect_ratio=:equal,
7                   title="Earth → Moon L1 ")
8      scatter!([-μ],      [0], marker=:circle, markersize=10, label="Earth")
9      annotate!([-μ],      [0.05], text("Earth\\nx = $(round(-μ, digits=6))", :blue,
:      :center, 10))
10     scatter!([1-μ],      [0], marker=:star5, markersize=12, label="Moon")
11     annotate!([1-μ],      [0.05], text("Moon\\nx = $(round(1-μ, digits=6))", :black,
:      :center, 10))
12     scatter!([x_L1],      [0], marker=:diamond, markersize=10, color=:red, label="L1")
13     annotate!([x_L1],      [0.05], text("L1\\nx = $(round(x_L1, digits=6))", :red,
:      :center, 10))
14
15     # Time histories
16     p_states = plot(ts, [opt_x opt_y], label=["x" "y"], title="Position vs Time")
17     p_vel     = plot(ts, [opt_vx opt_vy], label=["vx" "vy"], title="Velocity vs Time")
18     p_ctrl    = plot(ts, [opt_ux opt_uy], label=["ux" "uy"], title="Control vs Time",
19                     ylim=(-0.11,0.11))
20
21     display(p_traj)
22     display(p_states)
23     display(p_vel)
24     display(p_ctrl)
25 end

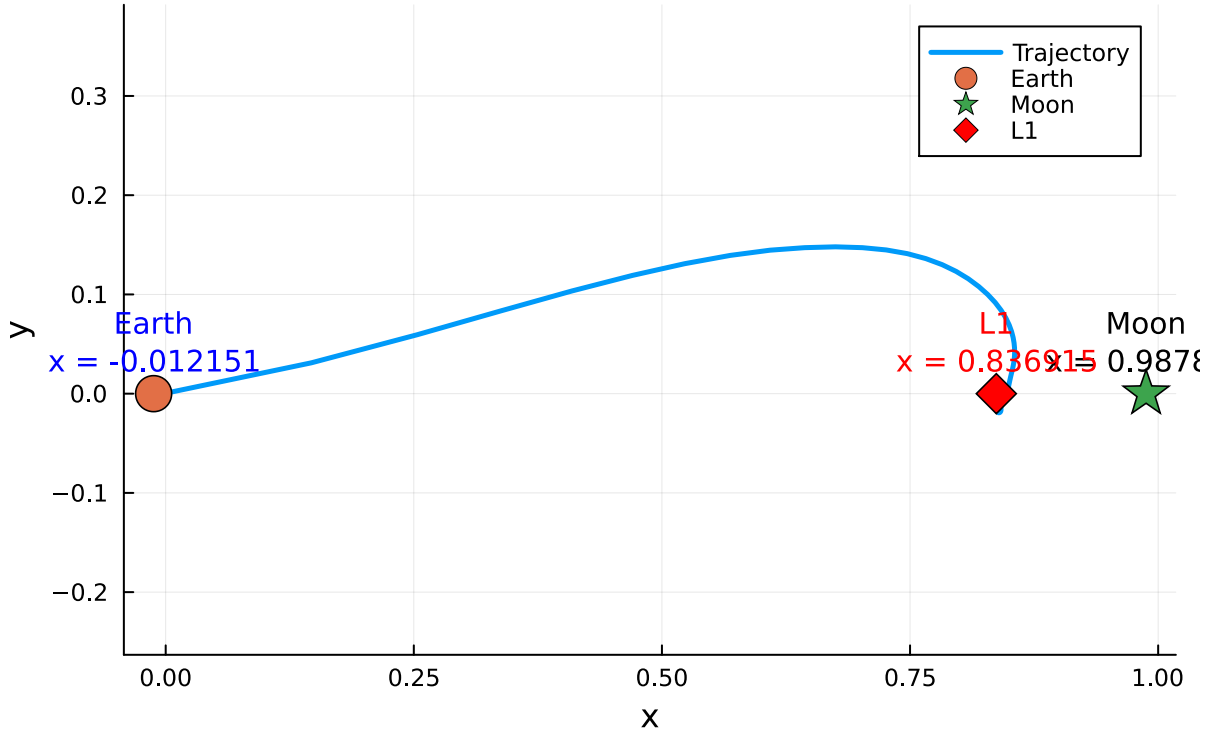
```

```

Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=2}
Plot{Plots.GRBackend() n=2}
Plot{Plots.GRBackend() n=2}

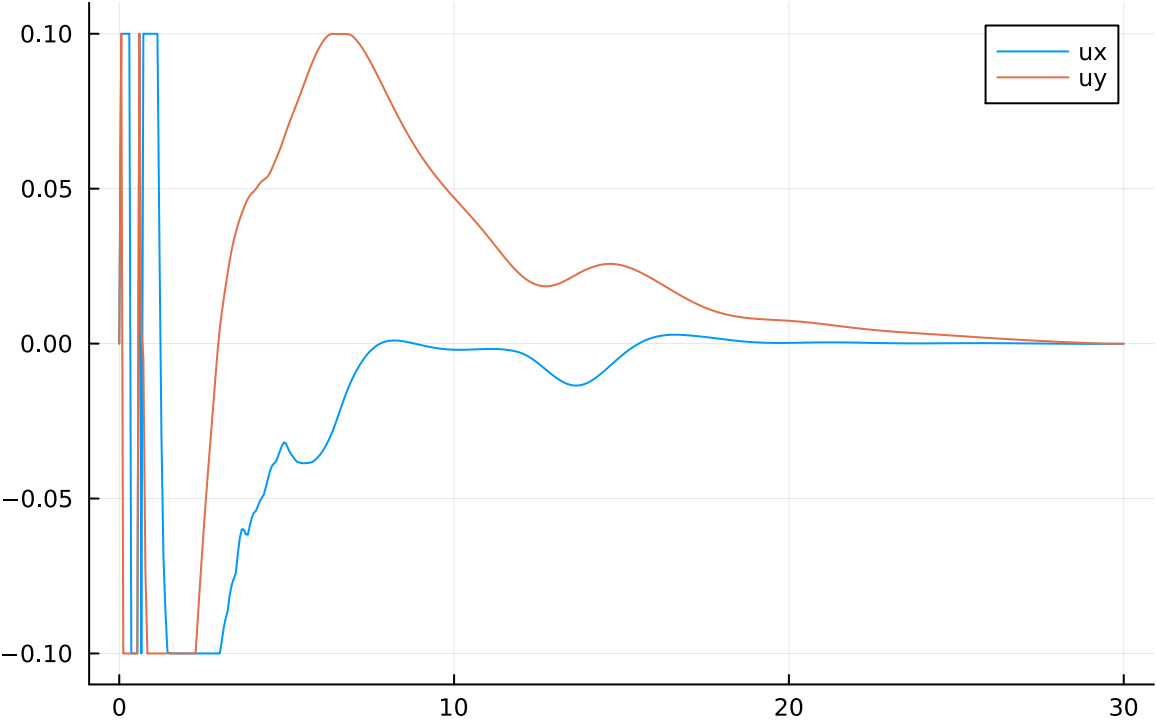
```

Earth → Moon L1



1 `p_traj`

Control vs Time



1 p_ctrl